Refinement and Type Difference in SKJS

Abstract

We formalize the internal type representation used by SKJS and study two related type transformations: (1) predicate-driven *refinement* (control-flow narrowing), and (2) the type-difference operation (diff) used during type inference. We state and prove soundness properties for these transformations, while explicitly documenting the intended sources of incompleteness.

1 Definitions Index

Symbol	Defined In
Tags	Definition 1
Payload, Val, rtag	Definition 2
SType	Definition 3
tag	Definition 4
A0	Definition 5
A1	Definition 6
STypes, tags$(\cdot)$	Definition 8
STC$(\cdot,\cdot)$	Definition 32
TInst	Definition 9
TInsts	Definition 10
TParam	Definition 11
Clause	Definition 12
Clauses	Definition 13
TType	Definition 14
refine (predicate-driven refinement)	Definition 15
diff (soundness P1/P2)	Definition 27

2 Runtime Model and Semantic Interface

Definition 1 (Runtime Tags)**.** *Let* Tags *be the set of runtime tags (the coarse runtime kind of a value). In the current implementation, tags are represented as integers, so one may take* $\mathsf{Tags} = \{0, 1, \ldots, 11\}$.

Definition 2 (Runtime Values and Tags)**.** *Assume runtime values carry their tag as part of the representation: let* Payload *be an abstract set of payloads and define*

$$\mathsf{Val} = \mathsf{Tags} \times \mathsf{Payload}.$$

Define the runtime tag function $\mathsf{rtag} : \mathsf{Val} \to \mathsf{Tags}$ *as the first projection:*

$$\mathsf{rtag}((k, p)) = k.$$

Definition 3 (Simple Types)**.** *Let* SType *be the set of* simple types*, i.e. the atomic type forms that have a single runtime tag. Formally,* SType *is the smallest set generated by the following constructors:*

- *primitive atoms:* $\texttt{undefined}$, $\texttt{void}$, $\texttt{null}$, $\texttt{boolean}$, $\texttt{number}$, $\texttt{string}$, $\texttt{bigint}$
- *boolean literals:* $\texttt{true}$, $\texttt{false}$
- *numeric literals:* n *for* $n \in \mathbb{R}$
- *string literals: "*x*" for strings* x
- *structured atoms:* $\texttt{tuple}(\ldots)$, $\texttt{Promise}(\ldots)$, $\texttt{object}(\ldots)$, $\texttt{algebraic}(\ldots)$ *(tagged unions/ADTs with a runtime discriminant),* $\texttt{function}(\ldots)$

We write these constructors suggestively; most internal parameters are left abstract. However, for control-flow refinement we will require a concrete semantic interface for object shapes (fields and index signatures); see Definition 7.

Definition 4 (Simple Types and Tags)**.** *The function* $\mathsf{tag} : \mathsf{SType} \to \mathsf{Tags}$ *assigns tags:*

$$\mathsf{tag}(\texttt{undefined}) = 0 \qquad \mathsf{tag}(\texttt{void}) = 1$$
$$\mathsf{tag}(\texttt{null}) = 2 \qquad \mathsf{tag}(\texttt{boolean}), \mathsf{tag}(\texttt{true}), \mathsf{tag}(\texttt{false}) = 3$$
$$\mathsf{tag}(\texttt{number}), \mathsf{tag}(n) = 4 \text{ for } n \in \mathbb{R} \qquad \mathsf{tag}(\texttt{string}), \mathsf{tag}(\text{“}x\text{”}) = 5$$
$$\mathsf{tag}(\texttt{bigint}) = 6 \qquad \mathsf{tag}(\texttt{tuple}(\ldots)) = 7$$
$$\mathsf{tag}(\texttt{Promise}(\ldots)) = 8 \qquad \mathsf{tag}(\texttt{object}(\ldots)) = 9$$
$$\mathsf{tag}(\texttt{algebraic}(\ldots)) = 10 \qquad \mathsf{tag}(\texttt{function}(\ldots)) = 11$$

The semantic interpretation $[\![\cdot]\!] : \mathsf{SType} \to \mathcal{P}(\mathsf{Val})$ *maps each simple type to its set of inhabitants.*

Note. *We do* not *define* $[\![\cdot]\!]$ *in this document; it is treated as an abstract interpretation function. Instead, we state explicit assumptions about its behavior as axioms (A0–A1).*

Definition 5 (**A0** (Runtime Tag Consistency))**.** *The semantic interpretation of simple types agrees with runtime tags:*

$$\forall s \in \mathsf{SType}.\ \forall v \in \mathsf{Val}.\ v \in [\![s]\!] \implies \mathsf{rtag}(v) = \mathsf{tag}(s).$$

Definition 6 (**A1** (Tag Disjointness))**.**

$$\forall s_1, s_2 \in \mathsf{SType}.\ \mathsf{tag}(s_1) \neq \mathsf{tag}(s_2) \implies [\![s_1]\!] \cap [\![s_2]\!] = \emptyset$$

Definition 7 (Object Values and Object-Type Semantics)**.** *Let* $\mathsf{ObjVal} \subseteq \mathsf{Val}$ *be the set of object-tagged runtime values:*

$$\mathsf{ObjVal} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 9\}.$$

Assume a (partial) finite-property interface on objects:

$$\mathsf{props} : \mathsf{ObjVal} \to \mathcal{P}_{\mathrm{fin}}(\mathsf{String})$$
$$\mathsf{get} : \mathsf{ObjVal} \times \mathsf{String} \rightharpoonup \mathsf{Val}$$

such that $\mathsf{get}(o, p)$ *is defined exactly when* $p \in \mathsf{props}(o)$*. Write* $\mathsf{has}(o, p)$ *for* $p \in \mathsf{props}(o)$*.*

Object field specifications. *An* object field specification *is a triple* $F = (F_{\text{req}}, F_{\text{opt}}, F_{\text{idx}})$ *where:*

- $F_{\text{req}} : \mathsf{String} \rightharpoonup \mathsf{TType}$ *is a finite partial map of required fields,*
- $F_{\text{opt}} : \mathsf{String} \rightharpoonup \mathsf{TType}$ *is a finite partial map of optional fields,*
- F_{idx} *is either* None *or* $\mathsf{Some}(K, V)$ *for* $K, V \in \mathsf{TType}$ *(an index signature).*

Semantics of `object`(F)**.** *We assume the object constructor in* SType *can carry such an* F*, and define:*

$$o \in [\![\texttt{object}(F)]\!]$$

if and only if $o \in \mathsf{ObjVal}$ *and the following constraints hold:*

- ***Required fields:*** *for every* $p \in \mathrm{dom}(F_{\text{req}})$*,* $\mathsf{has}(o, p)$ *and* $\mathsf{get}(o, p) \in [\![F_{\text{req}}(p)]\!]$*.*
- ***Optional fields:*** *for every* $p \in \mathrm{dom}(F_{\text{opt}})$*, if* $\mathsf{has}(o, p)$ *then* $\mathsf{get}(o, p) \in [\![F_{\text{opt}}(p)]\!]$ *(and if not, no constraint).*
- ***Index signature (open objects):*** *if* $F_{\text{idx}} = \mathsf{Some}(K, V)$ *then for every* $p \in \mathsf{props}(o)$*: if the runtime string value "*p*" is in* $[\![K]\!]$*, then* $\mathsf{get}(o, p) \in [\![V]\!]$*.*

Note. *This semantics is intentionally* open*: objects may have properties not mentioned in* F_{req} *or* F_{opt}*.*

3 Internal Type Representation

Definition 8 (STypes)**.** STypes *is a finite partial map* $\mathsf{Tags} \rightharpoonup \mathsf{SType}$ *with at most one entry per tag: if* $S(k) = s$ *then* $\mathsf{tag}(s) = k$*.*

Partial-map conventions. *We write* $\mathrm{dom}(S)$ *for the domain of defined keys. When a definition returns "undefined" at a key* k*, that means precisely* $k \notin \mathrm{dom}(S)$ *(i.e. the partial map has no entry at* k*).*

$$[\![S]\!] = \bigcup_{k \in \mathrm{dom}(S)} [\![S(k)]\!]$$

We write $\mathsf{tags}(S) = \mathrm{dom}(S)$*. We write* $\{k \mapsto s\}$ *for the singleton partial map with domain* $\{k\}$*.*

Intuition. *An* STypes *value represents the* tag-level *part of a union type: it records, for each runtime tag* k *that may occur, a single representative simple type* s *of that tag. The map shape enforces a normalization invariant ("at most one entry per tag"), which matches the implementation: many operations reason by tags and intentionally do not keep multiple alternatives of the same tag.*

Example. *The union* `number | string` *is represented as:*

$$S = \{4 \mapsto \texttt{number},\ 5 \mapsto \texttt{string}\}$$

and has semantics $[\![S]\!] = [\![\texttt{number}]\!] \cup [\![\texttt{string}]\!]$*. In contrast,* `number` *and the numeric literal* `1` *share the same tag* 4*, so an* STypes *value cannot simultaneously store both; this is why, for example, a tag-based "diff" cannot precisely express* `number \ 1`*.*

Definition 9 (Type Instances)**.** *Let* TInst *be the set of type instances (type variables or generic instantiations).*

Examples. *A type variable instance is something like T (a unification variable introduced during type checking; this is* not *a source-level type parameter). A generic instantiation instance is a named type applied to type arguments, e.g.* `Promise`$\langle$`number`$\rangle$ *or* `Array`$\langle$`string`$\rangle$.

Each instance $i \in \mathsf{TInst}$ has a semantic interpretation $[\![i]\!] \subseteq \mathsf{Val}$.

Definition 10 (Type Instance Sets)**.** *Let $\mathsf{TInsts} = \mathcal{P}_{\text{fin}}(\mathsf{TInst})$ be the set of* finite *sets of instances. For $I \in \mathsf{TInsts}$:*

$$[\![I]\!] = \bigcup_{i \in I} [\![i]\!].$$

Definition 11 (Type Parameters)**.** *Let* TParam *be the set of* type parameters *(generic binders), i.e. names introduced by source-level generic declarations such as* `function f`$\langle T \rangle(\ldots)$ *or* `type Box`$\langle T \rangle = \ldots$. *They differ from type-variable instances: type parameters are binders whose meaning is fixed by the caller/instantiation, whereas type-variable instances are inference unknowns whose meaning is solved by the type checker.*

Definition 12 (Clause)**.** *A clause $c = (\mathsf{vars}, \mathsf{stypes})$ consists of:*

- $\mathsf{vars} \subseteq \mathsf{TParam}$*: a finite set of type parameters*
- $\mathsf{stypes} \in \{\mathsf{None}\} \cup \{\mathsf{Some}(S) \mid S \in \mathsf{STypes}\}$

Each type parameter $v \in \mathsf{TParam}$ has an interpretation $[\![v]\!] \subseteq \mathsf{Val}$. Semantics:

$$[\![(\mathsf{vars}, \mathsf{stypes})]\!] = \left(\bigcap_{v \in \mathsf{vars}} [\![v]\!] \right) \cap [\![\mathsf{stypes}]\!]$$

where $[\![\mathsf{None}]\!] = \mathsf{Val}$ (top) and $[\![\mathsf{Some}(S)]\!] = [\![S]\!]$. For $\mathsf{vars} = \emptyset$, $\bigcap_{v \in \emptyset} [\![v]\!] = \mathsf{Val}$.

Well-formedness: $\mathsf{stypes} = \mathsf{None} \implies \mathsf{vars} = \emptyset$ *(i.e., $(\emptyset, \mathsf{None})$ is the unique top clause).*

Definition 13 (Clauses)**.** Clauses *is a finite map from $\mathcal{P}_{\text{fin}}(\mathsf{TParam})$ to* Clause*, with at most one clause per key, and such that keys agree with clause variables: if $C(k) = (\mathsf{vars}, \mathsf{stypes})$ then $\mathsf{vars} = k$.*

$$[\![C]\!] = \bigcup_{k \in \mathrm{dom}(C)} [\![C(k)]\!]$$

Example. *Let $S_{\mathsf{num}} = \{4 \mapsto$* `number`$\}$ *and $S_{\mathsf{str}} = \{5 \mapsto$* `string`$\}$. *Let $A, B \in \mathsf{TParam}$ be two type parameters. Consider the clauses map C with two entries:*

$$C(\{A\}) = (\{A\}, \mathsf{Some}(S_{\mathsf{num}})) \qquad C(\{B\}) = (\{B\}, \mathsf{Some}(S_{\mathsf{str}})).$$

Then

$$[\![C]\!] = \big([\![A]\!] \cap [\![\texttt{number}]\!]\big) \cup \big([\![B]\!] \cap [\![\texttt{string}]\!]\big).$$

To illustrate a non-singleton key, consider instead the single-entry map C' defined by:

$$C'(\{A, B\}) = (\{A, B\}, \mathsf{Some}(S_{\mathsf{num}})).$$

Then $[\![C']\!] = [\![A]\!] \cap [\![B]\!] \cap [\![$`number`*$]\!]$, i.e. the clause requires* both *parameters.*

Definition 14 (TType)**.** $\mathsf{TType} = (\mathsf{insts}, \mathsf{clauses})$ *where* $\mathsf{insts} \in \mathsf{TInsts}$ *and* $\mathsf{clauses} \in \mathsf{Clauses}$.

$$[\![(\mathsf{insts}, \mathsf{clauses})]\!] = [\![\mathsf{insts}]\!] \cup [\![\mathsf{clauses}]\!]$$

Schematic. *It is helpful to keep the following picture in mind:*

$$\underbrace{(\mathsf{insts}, \mathsf{clauses})}_{\mathsf{TType}} \quad = \quad \underbrace{\mathsf{insts}}_{\text{“instance” part}} \quad \cup \quad \underbrace{\mathsf{clauses}}_{\text{“conditional” part}}$$

and later $\mathsf{diff}_{\mathsf{TType}}$ *simply applies* diff *componentwise to these two parts.*

Example. *Let* $I = \{\mathit{Promise}\langle \mathit{number}\rangle\} \in \mathsf{TInsts}$ *and let* C *be the clauses map from the previous example. Then* $t = (I, C)$ *denotes the type whose semantics is:*

$$[\![t]\!] = [\![\mathit{Promise}\langle \mathit{number}\rangle]\!] \cup [\![C]\!].$$

4 Refinement (Predicate-Driven Narrowing)

Definition 15 (Refinement)**.** *Let* $t \in \mathsf{TType}$ *and let* $P \in \mathcal{P}(\mathsf{Val})$ *be a predicate (a set of runtime values). A* refinement operator *is a function*

$$\mathsf{refine} : \mathsf{TType} \times \mathcal{P}(\mathsf{Val}) \to \mathsf{TType}.$$

We call refine branch-sound *if for all* $t \in \mathsf{TType}$ *and* $P \in \mathcal{P}(\mathsf{Val})$*:*

$$[\![t]\!] \cap P \subseteq [\![\mathsf{refine}(t, P)]\!].$$

Remark (precision vs soundness). *Branch-soundness only requires that the refined type* over-approximates *the values that can occur in the branch. It does not require* $[\![\mathsf{refine}(t, P)]\!] \subseteq [\![t]\!]$*, so the refinement is allowed to be conservative (lose information) while remaining sound.*

Implementation correspondence. In SKJS, control-flow narrowing is implemented by rewriting the internal type for a local variable using a tag-indexed filter/map on simple types (see the `FilterType` transformer and `setLocal` in the implementation). This motivates defining refinements compositionally by lifting a transformation on SType values to STypes, Clause, Clauses, and finally TType.

Definition 16 (Filter/Map Refinement (Implementation Model))**.** *Let* $P \in \mathcal{P}(\mathsf{Val})$ *be a predicate. Let* $f : \mathsf{SType} \to \{\mathsf{true}, \mathsf{false}\}$ *be a filter and* $g : \mathsf{SType} \to \mathsf{SType}$ *be a mapper. We say* (f, g) *is* sound for P *if for every* $s \in \mathsf{SType}$ *and every* $v \in [\![s]\!]$*:*

$$v \in P \implies (f(s) = \mathsf{true} \ \wedge \ v \in [\![g(s)]\!]).$$

Remark (tag preservation). *In the implementation, the mapper* g *used for refinement preserves runtime tags:* $\mathsf{tag}(g(s)) = \mathsf{tag}(s)$*. We do not require this for the semantic statement below, but it ensures the lifted* STypes *transformation does not merge distinct tags.*

Lift to STypes. *Define:*

$$\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S) \triangleq \{\, \mathsf{tag}(g(S(k))) \mapsto g(S(k)) \mid k \in \mathrm{dom}(S),\ f(S(k)) = \mathsf{true} \,\}.$$

Operationally, this is exactly the “filter-map” over the unique representative per tag.

Lift to Clause. *For a clause* $c = (\mathsf{vars}, \mathsf{stypes})$ *define:*

$$\mathsf{refine}^{f,g}_{\mathsf{Clause}}(c) \triangleq \begin{cases} (\mathsf{vars}, \mathsf{None}) & \text{if } \mathsf{stypes} = \mathsf{None} \\ (\mathsf{vars}, \mathsf{Some}(\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S))) & \text{if } \mathsf{stypes} = \mathsf{Some}(S) \end{cases}$$

and interpret the special case $\mathsf{Some}(\emptyset)$ *as* $\bot$ *(dropped), matching the implementation invariant that bottom clauses are not stored.*

Lift to Clauses. *For* $C \in \mathsf{Clauses}$*, define* $\mathsf{refine}^{f,g}_{\mathsf{Clauses}}(C)$ *by applying* $\mathsf{refine}^{f,g}_{\mathsf{Clause}}$ *to every clause and dropping* $\bot$ *results.*

Lift to TType. *For* $t = (\mathsf{insts}, \mathsf{clauses})$ *define:*

$$\mathsf{refine}^{f,g}(t) \triangleq (\mathsf{insts}, \mathsf{refine}^{f,g}_{\mathsf{Clauses}}(\mathsf{clauses})).$$

This matches the current implementation: refinement rewrites the clause part but does not filter the instance part.

Lemma 17 (Branch-Soundness of Filter/Map Refinement)**.** *If* (f, g) *is sound for* P*, then for all* $t \in \mathsf{TType}$*:*

$$[\![t]\!] \cap P \subseteq [\![\mathsf{refine}^{f,g}(t)]\!].$$

Proof. Let $t = (\mathsf{insts}, \mathsf{clauses})$. We prove $[\![t]\!] \cap P \subseteq [\![\mathsf{refine}^{f,g}(t)]\!]$ by cases on membership in $[\![\mathsf{insts}]\!] \cup [\![\mathsf{clauses}]\!]$.

- If $v \in [\![\mathsf{insts}]\!] \cap P$, then $v \in [\![\mathsf{insts}]\!] \subseteq [\![\mathsf{refine}^{f,g}(t)]\!]$ since refinement leaves insts unchanged.
- If $v \in [\![\mathsf{clauses}]\!] \cap P$, then $v \in [\![(\mathsf{vars}, \mathsf{stypes})]\!]$ for some clause $(\mathsf{vars}, \mathsf{stypes})$ in clauses. If $\mathsf{stypes} = \mathsf{None}$, then $\mathsf{refine}^{f,g}_{\mathsf{Clause}}(\mathsf{vars}, \mathsf{None}) = (\mathsf{vars}, \mathsf{None})$ and the clause remains top, so v is still included. If $\mathsf{stypes} = \mathsf{Some}(S)$, then $v \in [\![S(k)]\!]$ for some $k \in \mathrm{dom}(S)$. By soundness of (f, g) for P and $v \in P$, we have $f(S(k)) = \mathsf{true}$ and $v \in [\![g(S(k))]\!]$. Hence $v \in [\![\mathsf{refine}^{f,g}_{\mathsf{STypes}}(S)]\!]$, so v is contained in the refined clause and therefore in $[\![\mathsf{refine}^{f,g}_{\mathsf{Clauses}}(\mathsf{clauses})]\!]$.

Combining cases yields the desired inclusion. □

4.1 Algorithmic Refinement Judgements (Inference-Rule Style)

To present refinement in the idiomatic style used in type-system research, we write judgements that update a typing environment. Let Γ range over environments mapping local variables to TType. We write $\Gamma(x)$ for lookup and $\Gamma[x \mapsto t]$ for pointwise update.

True/false refinements. We define two refinement judgements:

$$\Gamma \vdash e \Rightarrow \Gamma' \qquad \text{and} \qquad \Gamma \vdash e \Leftarrow \Gamma'$$

to mean "refine under the assumption that e evaluates to `true`/`false`" respectively. In this note we only give rules for the refinement forms implemented in SKJS; all other expressions leave the environment unchanged.

Local update helper. Given a local x and a refinement transformer $R : \mathsf{TType} \to \mathsf{TType}$, define:

$$\mathsf{upd}(\Gamma, x, R) \triangleq \Gamma[x \mapsto R(\Gamma(x))].$$

When a rule refines a non-local expression, it produces Γ.

$$\frac{}{\Gamma \vdash e \Rightarrow \Gamma}\ \text{R-OtherT} \qquad \frac{}{\Gamma \vdash e \Leftarrow \Gamma}\ \text{R-OtherF}$$

Nullish equality.

$$\frac{}{\Gamma \vdash (x \texttt{ === null}) \Rightarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_{\mathsf{null}}, g_{\mathsf{null}}}(t))}\ \text{R-NullT}$$

$$\frac{}{\Gamma \vdash (x \texttt{ === null}) \Leftarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_{\neg\mathsf{null}}, g_{\mathsf{null}}}(t))}\ \text{R-NullF}$$

$$\frac{}{\Gamma \vdash (x \texttt{ === undefined}) \Rightarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_{\mathsf{undef}}, g_{\mathsf{undef}}}(t))}\ \text{R-UndefT}$$

$$\frac{}{\Gamma \vdash (x \texttt{ === undefined}) \Leftarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_{\neg\mathsf{undef}}, g_{\mathsf{undef}}}(t))}\ \text{R-UndefF}$$

`typeof`.

$$\frac{}{\Gamma \vdash (\texttt{typeof } x \texttt{ === } k) \Rightarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_{P_k}, g_{P_k}}(t))}\ \text{R-TypeofT}$$

$$\frac{}{\Gamma \vdash (\texttt{typeof } x \texttt{ === } k) \Leftarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{\neg f_{P_k}, g_{P_k}}(t))}\ \text{R-TypeofF}$$

where P_k is the corresponding predicate from Definition 25.

`in`.

$$\frac{}{\Gamma \vdash (\texttt{"}p\texttt{" in } x) \Rightarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}(t))}\ \text{R-InT}$$

$$\frac{}{\Gamma \vdash (\texttt{"}p\texttt{" in } x) \Leftarrow \mathsf{upd}(\Gamma, x,\ t \mapsto \mathsf{refine}^{f_p^{\mathsf{false}}, g_p^{\mathsf{false}}}(t))}\ \text{R-InF}$$

Boolean connectives. The implementation composes refinements sequentially. For example:

$$\frac{\Gamma \vdash e_1 \Rightarrow \Gamma_1 \qquad \Gamma_1 \vdash e_2 \Rightarrow \Gamma_2}{\Gamma \vdash (e_1 \texttt{ \&\& } e_2) \Rightarrow \Gamma_2}\ \text{R-AndT} \qquad \frac{\Gamma \vdash e_1 \Leftarrow \Gamma_1 \qquad \Gamma_1 \vdash e_2 \Leftarrow \Gamma_2}{\Gamma \vdash (e_1 \texttt{ || } e_2) \Leftarrow \Gamma_2}\ \text{R-OrF}$$

Soundness connection. Each of the case-study lemmas in the following subsections proves that the corresponding rule is branch-sound, via Lemma 17.

4.2 Refinement Case Study: the `in` Operator

We formalize the refinement performed for conditions of the form `"p" in x` in SKJS. The implementation refines only local variables and updates object types by either (a) adding a required field on the true branch, or (b) forgetting a field on the false branch.

Definition 18 (`in` Predicates)**.** *Fix a property name $p \in \mathsf{String}$. Define:*

$$P_{\text{in}}^{\mathsf{true}}(p) \triangleq \{\, o \in \mathsf{Val} \mid o \in \mathsf{ObjVal} \wedge \mathsf{has}(o, p) \,\} \qquad P_{\text{in}}^{\mathsf{false}}(p) \triangleq \mathsf{Val} \setminus P_{\text{in}}^{\mathsf{true}}(p).$$

Definition 19 (`in` Filter/Map Pair)**.** *Fix $p \in \mathsf{String}$ and let $\top_{\mathsf{TType}}$ be any type with $[\![\top_{\mathsf{TType}}]\!] = \mathsf{Val}$ (e.g. the unique top clause). Define a mapper $g_p^{\mathsf{true}} : \mathsf{SType} \to \mathsf{SType}$ as follows:*

- *If s is an object type $s = \texttt{object}(F)$ and $F_{\text{idx}} = \mathsf{None}$ and $p \notin \mathrm{dom}(F_{\text{req}})$, then $g_p^{\mathsf{true}}(s)$ is $\texttt{object}(F')$ where F' is F except that F'_{req} additionally maps $p \mapsto \top_{\mathsf{TType}}$.*
- *Otherwise, $g_p^{\mathsf{true}}(s) = s$.*

Let $f_p^{\mathsf{true}} : \mathsf{SType} \to \{\mathsf{true}, \mathsf{false}\}$ be the filter that keeps object-tagged structured types and drops primitive atoms and primitive values (matching the implementation's conservative approximation); for the purpose of this note it suffices that:

$$f_p^{\mathsf{true}}(s) = \mathsf{false} \text{ for primitive/unit/value } s, \quad \text{and} \quad f_p^{\mathsf{true}}(\texttt{object}(F)) = \mathsf{true}.$$

Define a mapper $g_p^{\mathsf{false}} : \mathsf{SType} \to \mathsf{SType}$ by:

- *If $s = \texttt{object}(F)$ then $g_p^{\mathsf{false}}(s) = \texttt{object}(F \setminus p)$ where $F \setminus p$ removes p from both F_{req} and F_{opt} (and leaves the index signature unchanged).*
- *Otherwise $g_p^{\mathsf{false}}(s) = s$.*

Let $f_p^{\mathsf{false}}(s) = \mathsf{true}$ for all s (the implementation does not attempt to exclude non-objects in the false branch).

Lemma 20 (`in`-True Branch is Branch-Sound)**.** *For every $p \in \mathsf{String}$, the refinement $\mathsf{refine}^{f_p^{\mathsf{true}}, g_p^{\mathsf{true}}}$ is branch-sound for $P_{\text{in}}^{\mathsf{true}}(p)$.*

Proof. We show that $(f_p^{\mathsf{true}}, g_p^{\mathsf{true}})$ is sound for $P_{\text{in}}^{\mathsf{true}}(p)$ in the sense of Definition 16, and conclude by Lemma 17. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\text{in}}^{\mathsf{true}}(p)$. Then $v \in \mathsf{ObjVal}$, so $\mathsf{rtag}(v) = 9$. By A0, $\mathsf{rtag}(v) = \mathsf{tag}(s)$, hence $\mathsf{tag}(s) = 9$ and therefore s must be an object type of the form $\texttt{object}(F)$. By definition of $P_{\text{in}}^{\mathsf{true}}(p)$, $\mathsf{has}(v, p)$. The filter keeps object types, so $f_p^{\mathsf{true}}(s) = \mathsf{true}$. If g_p^{true} adds the required field $p \mapsto \top_{\mathsf{TType}}$, then v satisfies the added requirement because it already has p and $[\![\top_{\mathsf{TType}}]\!] = \mathsf{Val}$. Otherwise $g_p^{\mathsf{true}}(s) = s$ and membership is immediate. Thus $v \in [\![g_p^{\mathsf{true}}(s)]\!]$ and soundness holds. □

Lemma 21 (`in`-False Branch is Branch-Sound)**.** *For every $p \in \mathsf{String}$, the refinement $\mathsf{refine}^{f_p^{\mathsf{false}}, g_p^{\mathsf{false}}}$ is branch-sound for $P_{\text{in}}^{\mathsf{false}}(p)$.*

Proof. As above, it suffices to show $(f_p^{\mathsf{false}}, g_p^{\mathsf{false}})$ is sound for $P_{\text{in}}^{\mathsf{false}}(p)$. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\text{in}}^{\mathsf{false}}(p)$. We have $f_p^{\mathsf{false}}(s) = \mathsf{true}$ by definition. If s is not an object type, then $g_p^{\mathsf{false}}(s) = s$ so $v \in [\![g_p^{\mathsf{false}}(s)]\!]$. If $s = \texttt{object}(F)$, then $g_p^{\mathsf{false}}(s) = \texttt{object}(F \setminus p)$ which removes constraints. By Definition 7, dropping field constraints can only enlarge the set of inhabitants, so $[\![\texttt{object}(F)]\!] \subseteq [\![\texttt{object}(F \setminus p)]\!]$ and hence $v \in [\![g_p^{\mathsf{false}}(s)]\!]$. □

4.3 Refinement Case Study: Nullish Checks

We model the refinements performed for comparisons of a local variable against `null` and `undefined`. In the implementation, `undefined` and `void` are treated equivalently for refinement.

Definition 22 (Nullish Predicates)**.** *Define the following predicates on runtime values:*

$$P_{\mathsf{null}} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 2\} \qquad P_{\mathsf{undef}} \triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) \in \{0,1\}\}.$$

We also write $\neg P$ *for* $\mathsf{Val} \setminus P$*.*

Lemma 23 (Null Refinement is Branch-Sound)**.** *Let* $f_{\mathsf{null}}(s) \triangleq (\mathsf{tag}(s) = 2)$ *and let* $g_{\mathsf{null}}(s) \triangleq s$*. Then* $\mathsf{refine}^{f_{\mathsf{null}}, g_{\mathsf{null}}}$ *is branch-sound for* P_{null}*. Moreover, letting* $f_{\neg\mathsf{null}}(s) \triangleq (\mathsf{tag}(s) \neq 2)$*,* $\mathsf{refine}^{f_{\neg\mathsf{null}}, g_{\mathsf{null}}}$ *is branch-sound for* $\neg P_{\mathsf{null}}$*.*

Proof. We prove soundness of $(f_{\mathsf{null}}, g_{\mathsf{null}})$ for P_{null} and apply Lemma 17. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P_{\mathsf{null}}$. Then $\mathsf{rtag}(v) = 2$, and by A0 we have $\mathsf{tag}(s) = 2$, hence $f_{\mathsf{null}}(s) = \mathsf{true}$. Since g_{null} is the identity, $v \in [\![g_{\mathsf{null}}(s)]\!]$ is immediate. The $\neg P_{\mathsf{null}}$ case is analogous. □

Lemma 24 (Undefined/Void Refinement is Branch-Sound)**.** *Let* $f_{\mathsf{undef}}(s) \triangleq (\mathsf{tag}(s) \in \{0,1\})$ *and* $g_{\mathsf{undef}}(s) \triangleq s$*. Then* $\mathsf{refine}^{f_{\mathsf{undef}}, g_{\mathsf{undef}}}$ *is branch-sound for* P_{undef}*. Moreover, letting* $f_{\neg\mathsf{undef}}(s) \triangleq (\mathsf{tag}(s) \notin \{0,1\})$*,* $\mathsf{refine}^{f_{\neg\mathsf{undef}}, g_{\mathsf{undef}}}$ *is branch-sound for* $\neg P_{\mathsf{undef}}$*.*

Proof. As above, we show $(f_{\mathsf{undef}}, g_{\mathsf{undef}})$ is sound for P_{undef}. Let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $\mathsf{rtag}(v) \in \{0,1\}$. By A0, $\mathsf{tag}(s) = \mathsf{rtag}(v) \in \{0,1\}$, so $f_{\mathsf{undef}}(s) = \mathsf{true}$ and $v \in [\![g_{\mathsf{undef}}(s)]\!]$. The $\neg P_{\mathsf{undef}}$ case is analogous. □

4.4 Refinement Case Study: `typeof` Checks

We model the refinements performed for tests of the form `typeof x === "k"` for a subset of strings k. The implementation refines based on the internal tag/classification of the current SType alternatives.

Note. The predicate modeled here is the one used by the checker for narrowing; it may be a conservative approximation of JavaScript runtime `typeof`.

Definition 25 (`typeof` Predicates (Tag-Level))**.** *Define the following tag-level predicates:*

$$\begin{aligned}
P_{\mathsf{typeof}=\texttt{"string"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 5\} \\
P_{\mathsf{typeof}=\texttt{"number"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 4\} \\
P_{\mathsf{typeof}=\texttt{"bigint"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 6\} \\
P_{\mathsf{typeof}=\texttt{"boolean"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 3\} \\
P_{\mathsf{typeof}=\texttt{"function"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) = 11\} \\
P_{\mathsf{typeof}=\texttt{"undefined"}} &\triangleq \{v \in \mathsf{Val} \mid \mathsf{rtag}(v) \in \{0,1\}\}.
\end{aligned}$$

Lemma 26 (`typeof` Refinement is Branch-Sound (Tag-Level))**.** *For each predicate* P *in Definition 25, define* $f_P(s) \triangleq (\mathsf{tag}(s) \in \mathsf{tags}_P)$ *where* tags_P *is the corresponding set of tags (e.g.* $\{5\}$ *for* `"string"`*,* $\{0,1\}$ *for* `"undefined"`*), and let* $g_P(s) \triangleq s$*. Then* $\mathsf{refine}^{f_P, g_P}$ *is branch-sound for* P*, and* $\mathsf{refine}^{\neg f_P, g_P}$ *is branch-sound for* $\neg P$*.*

Proof. Fix P and let $s \in \mathsf{SType}$ and $v \in [\![s]\!]$ with $v \in P$. By A0, $\mathsf{tag}(s) = \mathsf{rtag}(v) \in \mathsf{tags}_P$, so $f_P(s) = \mathsf{true}$ and $v \in [\![g_P(s)]\!]$. Apply Lemma 17. The $\neg P$ case is analogous. □

5 Type Difference (Diff)

Definition 27 (Diff Soundness Properties)**.** *Let X be a set equipped with a semantic interpretation $[\![\cdot]\!] : X \to \mathcal{P}(\mathsf{Val})$. An operation $\mathsf{diff} : X \times X \to X$ is* sound *(with respect to $[\![\cdot]\!]$) if for all $x_1, x_2 \in X$, letting $x_3 = \mathsf{diff}(x_1, x_2)$, the following properties hold:*

$$\textit{(P1)} \quad [\![x_3]\!] \subseteq [\![x_1]\!] \qquad \text{(under-approximation)}$$
$$\textit{(P2)} \quad [\![x_3]\!] \cup [\![x_2]\!] \supseteq [\![x_1]\!] \qquad \text{(coverage)}$$

In this document we instantiate X with STypes*,* Clause*,* Clauses*, and* TType*.*

Reader's guide. *These two properties imply that $[\![x_3]\!]$ is always a refinement of $[\![x_1]\!]$, and that it still covers all of $[\![x_1]\!]$ once $[\![x_2]\!]$ is added back. Equivalently (Lemma 28), $[\![x_3]\!]$ is an* over-approximation *of the true set difference $[\![x_1]\!] \setminus [\![x_2]\!]$ while staying within $[\![x_1]\!]$.*

Lemma 28 (Equivalent Characterization)**.** *For all $x_1, x_2, x_3 \in X$, (P1) $\wedge$ (P2) holds if and only if*

$$[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!] \subseteq [\![x_1]\!].$$

Proof. ($\Rightarrow$) Let $v \in [\![x_1]\!] \setminus [\![x_2]\!]$. By (P2), $v \in [\![x_3]\!] \cup [\![x_2]\!]$. Since $v \notin [\![x_2]\!]$, we have $v \in [\![x_3]\!]$. Thus $[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!]$. Combined with (P1): $[\![x_1]\!] \setminus [\![x_2]\!] \subseteq [\![x_3]\!] \subseteq [\![x_1]\!]$.

($\Leftarrow$) The right-hand inclusion $[\![x_3]\!] \subseteq [\![x_1]\!]$ is exactly (P1). For (P2),

$$[\![x_3]\!] \cup [\![x_2]\!] \supseteq ([\![x_1]\!] \setminus [\![x_2]\!]) \cup [\![x_2]\!] \supseteq [\![x_1]\!].$$

□

Definition 29 (Completeness)**.** *A diff operation is* complete *if:*

$$[\![\mathsf{diff}(t_1, t_2)]\!] = [\![t_1]\!] \setminus [\![t_2]\!]$$

By the equivalent characterization, (P1) $\wedge$ (P2) implies $[\![t_1]\!] \setminus [\![t_2]\!] \subseteq [\![\mathsf{diff}(t_1, t_2)]\!]$. Completeness additionally requires the reverse: $[\![\mathsf{diff}(t_1, t_2)]\!] \subseteq [\![t_1]\!] \setminus [\![t_2]\!]$.

5.1 STypes Diff

Definition 30 (STypes Diff)**.**

$$\mathsf{diff}_{\mathsf{STypes}}(L, R)(k) = \begin{cases} L(k) & \text{if } k \in \mathsf{tags}(L) \setminus \mathsf{tags}(R) \\ \textit{undefined} & \textit{otherwise} \end{cases}$$

Intuition. *Since an* STypes *value stores at most one representative per runtime tag, this diff only removes whole tags: it preserves exactly the tags present on the left but absent on the right.*

Lemma 31 (STypes Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] \subseteq [\![L]\!]$.

Proof. By definition, $\mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L, R)) = \mathsf{tags}(L) \setminus \mathsf{tags}(R) \subseteq \mathsf{tags}(L)$, and for all $k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L, R))$ we have $\mathsf{diff}_{\mathsf{STypes}}(L, R)(k) = L(k)$. Therefore:

$$[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] = \bigcup_{k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L,R))} [\![L(k)]\!] \subseteq \bigcup_{k \in \mathsf{tags}(L)} [\![L(k)]\!] = [\![L]\!].$$

□

Definition 32 (Same-Tag Coverage Condition)**.** *For $L, R \in \mathsf{STypes}$, define:*

$$\mathsf{STC}(L, R) \iff \forall k \in \mathsf{tags}(L) \cap \mathsf{tags}(R).\ [\![L(k)]\!] \subseteq [\![R(k)]\!]$$

Lemma 33 (STypes Diff Satisfies (P2) under STC)**.** *If $\mathsf{STC}(L, R)$ holds, then $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] \cup [\![R]\!] \supseteq [\![L]\!]$.*

Proof. Let $x \in [\![L]\!]$. Then $x \in [\![L(k)]\!]$ for some $k \in \mathsf{tags}(L)$.

- Case $k \notin \mathsf{tags}(R)$: Then $k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L, R))$ and $\mathsf{diff}_{\mathsf{STypes}}(L, R)(k) = L(k)$, so $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!]$.
- Case $k \in \mathsf{tags}(R)$: By STC, $[\![L(k)]\!] \subseteq [\![R(k)]\!] \subseteq [\![R]\!]$, so $x \in [\![R]\!]$.

□

Corollary 34 (STypes Diff under STC)**.** *If $\mathsf{STC}(L, R)$ holds, then (P2) holds. In particular, if $L(k) = R(k)$ for all $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$, then (P2) holds.*

Lemma 35 (STypes Diff Completeness)**.** $\mathsf{diff}_{\mathsf{STypes}}$ *is complete (i.e., $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$) if and only if $\mathsf{STC}(L, R)$.*

Proof. ($\Rightarrow$) Suppose completeness holds and let $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$. Since $k \notin \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L, R))$, the component $L(k)$ contributes no value to $[\![L]\!] \setminus [\![R]\!]$ under completeness, hence $[\![L(k)]\!] \subseteq [\![R]\!]$. By A1 and the "at most one entry per tag" invariant for STypes, $[\![L(k)]\!]$ can only overlap with $[\![R(k)]\!]$, so $[\![L(k)]\!] \subseteq [\![R(k)]\!]$.

($\Leftarrow$) We show $[\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$.

($\subseteq$) Let $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!]$. Then $x \in [\![L(k)]\!]$ for some $k \in \mathsf{tags}(L) \setminus \mathsf{tags}(R)$. By A1, $x \notin [\![R]\!]$ (no type in R has tag k), so $x \in [\![L]\!] \setminus [\![R]\!]$.

($\supseteq$) Let $x \in [\![L]\!] \setminus [\![R]\!]$. Then $x \in [\![L(k)]\!]$ for some $k \in \mathsf{tags}(L)$ and $x \notin [\![R]\!]$. If $k \in \mathsf{tags}(R)$, then by hypothesis $[\![L(k)]\!] \subseteq [\![R(k)]\!] \subseteq [\![R]\!]$, contradicting $x \notin [\![R]\!]$. So $k \notin \mathsf{tags}(R)$, meaning $k \in \mathsf{tags}(\mathsf{diff}_{\mathsf{STypes}}(L, R))$ and $x \in [\![\mathsf{diff}_{\mathsf{STypes}}(L, R)]\!]$. □

Corollary 36 (When STypes Diff is Incomplete)**.** $\mathsf{diff}_{\mathsf{STypes}}(L, R)$ *is incomplete when there exists $k \in \mathsf{tags}(L) \cap \mathsf{tags}(R)$ with $[\![L(k)]\!] \not\subseteq [\![R(k)]\!]$. Example:* $\mathsf{diff}_{\mathsf{STypes}}(\{4 \mapsto \texttt{number}\}, \{4 \mapsto \texttt{1}\}) = \emptyset$*, but $[\![\texttt{number}]\!] \setminus [\![\texttt{1}]\!] \neq \emptyset$.*

5.2 TInsts Diff

Definition 37 (TInsts Diff)**.**

$$\mathsf{diff}_{\mathsf{TInsts}}(L, R) = L \setminus R = \{i \in L \mid i \notin R\}$$

Note. *In the implementation,* TInsts *is represented as a collection of instance AST nodes and* $\mathsf{diff}_{\mathsf{TInsts}}$ *removes elements based on* structural equality *of instances. In this paper, that corresponds exactly to the usual mathematical set difference on the set* TInst *(i.e. membership/equality is the syntactic identity of instances, not semantic overlap).*

Lemma 38 (TInsts Diff Satisfies (P1) and (P2))**.** $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \subseteq [\![L]\!]$ *and* $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \cup [\![R]\!] \supseteq [\![L]\!]$.

Proof. (P1): $\mathsf{diff}_{\mathsf{TInsts}}(L, R) \subseteq L$, so $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] \subseteq [\![L]\!]$.

(P2): Let $x \in [\![L]\!]$. Then $x \in [\![i]\!]$ for some $i \in L$.

- If $i \in R$: Then $x \in [\![i]\!] \subseteq [\![R]\!]$.
- If $i \notin R$: Then $i \in \mathsf{diff}_{\mathsf{TInsts}}(L, R)$, so $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!]$.

□

Lemma 39 (TInsts Diff Completeness)**.** $\mathsf{diff}_{\mathsf{TInsts}}$ *is complete if and only if for all* $i \in L \setminus R$ *and* $j \in R$*:* $[\![i]\!] \cap [\![j]\!] = \emptyset$*.*

Proof. ($\Rightarrow$) Suppose completeness and let $i \in L \setminus R$ and $j \in R$. If $x \in [\![i]\!] \cap [\![j]\!]$, then $i \in \mathsf{diff}_{\mathsf{TInsts}}(L, R)$, so $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$. But $x \in [\![j]\!] \subseteq [\![R]\!]$, contradiction.

($\Leftarrow$) We show $[\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!] = [\![L]\!] \setminus [\![R]\!]$.

($\subseteq$) Let $x \in [\![\mathsf{diff}_{\mathsf{TInsts}}(L, R)]\!]$. Then $x \in [\![i]\!]$ for some $i \in L \setminus R$. For any $j \in R$, $[\![i]\!] \cap [\![j]\!] = \emptyset$ by hypothesis. Thus $x \notin [\![R]\!]$, so $x \in [\![L]\!] \setminus [\![R]\!]$.

($\supseteq$) Let $x \in [\![L]\!] \setminus [\![R]\!]$. Then $x \in [\![i]\!]$ for some $i \in L$. If $i \in R$, then $x \in [\![R]\!]$, contradiction. So $i \in L \setminus R = \mathsf{diff}_{\mathsf{TInsts}}(L, R)$. □

5.3 Clause Diff

Definition 40 (Clause Diff)**.** *For clauses* $c_1 = (\mathsf{vars}_1, \mathsf{stypes}_1)$ *and* $c_2 = (\mathsf{vars}_2, \mathsf{stypes}_2)$*:*

$$\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2) = \begin{cases} \bot & \text{if } \mathsf{stypes}_2 = \mathsf{None} \\ c_1 & \text{if } \mathsf{stypes}_1 = \mathsf{None} \\ (\mathsf{vars}_1, \mathsf{Some}(\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2))) & \text{if } \mathsf{stypes}_1 = \mathsf{Some}(S_1), \mathsf{stypes}_2 = \mathsf{Some}(S_2) \end{cases}$$

where $\bot$ *represents the empty type (dropped from results) with* $[\![\bot]\!] = \emptyset$*.*

Lemma 41 (Clause Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2)]\!] \subseteq [\![c_1]\!]$.

Proof. Case analysis:

- $\mathsf{stypes}_2 = \mathsf{None}$: Result is $\bot$. $[\![\bot]\!] = \emptyset \subseteq [\![c_1]\!]$.
- $\mathsf{stypes}_1 = \mathsf{None}$: Result is c_1. $[\![c_1]\!] \subseteq [\![c_1]\!]$.
- Both Some: $[\![(\mathsf{vars}_1, \mathsf{Some}(\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)))]\!] = \left(\bigcap_{v \in \mathsf{vars}_1} [\![v]\!]\right) \cap [\![\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)]\!] \subseteq \left(\bigcap_{v \in \mathsf{vars}_1} [\![v]\!]\right) \cap [\![S_1]\!] = [\![c_1]\!]$.

□

Lemma 42 (Clause Diff Satisfies (P2) Conditionally)**.** $\mathsf{diff}_{\mathsf{Clause}}$ *satisfies (P2) when:*

- *Case* $\mathsf{stypes}_2 = \mathsf{None}$*: Always for well-formed clauses (since then* c_2 *is the unique top clause and* $[\![c_2]\!] = \mathsf{Val} \supseteq [\![c_1]\!]$*).*
- *Case* $\mathsf{stypes}_1 = \mathsf{None}$*: Always (since result is* c_1*).*
- *Case both* Some*: When* $\mathsf{diff}_{\mathsf{STypes}}(S_1, S_2)$ *satisfies (P2).*

5.4 Clauses Diff

Definition 43 (Clauses Diff)**.** *For $C_1, C_2 \in \mathsf{Clauses}$, let $c_\emptyset = C_2(\emptyset)$ if $\emptyset \in \mathrm{dom}(C_2)$, else undefined.*

$$\mathsf{diff}_{\mathsf{Clauses}}(C_1, C_2)(k) = \begin{cases} \mathsf{diff}_{\mathsf{Clause}}(\mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k)), c_\emptyset) & \text{if } k \in \mathrm{dom}(C_2) \wedge c_\emptyset \text{ defined} \\ \mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k)) & \text{if } k \in \mathrm{dom}(C_2) \wedge c_\emptyset \text{ undefined} \\ \mathsf{diff}_{\mathsf{Clause}}(C_1(k), c_\emptyset) & \text{if } k \notin \mathrm{dom}(C_2) \wedge c_\emptyset \text{ defined} \\ C_1(k) & \text{if } k \notin \mathrm{dom}(C_2) \wedge c_\emptyset \text{ undefined} \end{cases}$$

for $k \in \mathrm{dom}(C_1)$. Clauses mapped to $\bot$ are dropped.

Note. *Whenever $k \in \mathrm{dom}(C_2)$, the call $\mathsf{diff}_{\mathsf{Clause}}(C_1(k), C_2(k))$ compares clauses stored under the* same key *k, so their variable-sets match by the* Clauses *well-formedness condition (keys agree with clause variables). The only subtraction that may use a different key is via $c_\emptyset = C_2(\emptyset)$, which is deliberately applied to every $C_1(k)$; this corresponds to the implementation behavior and explains why the right-hand clause's variables are not used by $\mathsf{diff}_{\mathsf{Clause}}$.*

Why the $\emptyset$-clause matters. *The distinguished key $\emptyset$ corresponds to clauses that do not require any type parameters. When present on the right, subtracting $c_\emptyset$ from every left clause models a "global" elimination that should apply regardless of which type-parameter constraints produced the left clause.*

Lemma 44 (Clauses Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{Clauses}}(C_1, C_2)]\!] \subseteq [\![C_1]\!]$.

Proof. Each output clause is derived from $C_1(k)$ via $\mathsf{diff}_{\mathsf{Clause}}$, which satisfies (P1). □

5.5 TType Diff

Definition 45 (TType Diff)**.** *For $t_1 = (\mathsf{insts}_1, \mathsf{clauses}_1)$ and $t_2 = (\mathsf{insts}_2, \mathsf{clauses}_2)$:*

$$\mathsf{diff}_{\mathsf{TType}}(t_1, t_2) = (\mathsf{diff}_{\mathsf{TInsts}}(\mathsf{insts}_1, \mathsf{insts}_2), \mathsf{diff}_{\mathsf{Clauses}}(\mathsf{clauses}_1, \mathsf{clauses}_2))$$

Theorem 46 (TType Diff Satisfies (P1))**.** $[\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] \subseteq [\![t_1]\!]$.

Proof.

$$\begin{aligned} [\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] &= [\![\mathsf{diff}_{\mathsf{TInsts}}(\mathsf{insts}_1, \mathsf{insts}_2)]\!] \cup [\![\mathsf{diff}_{\mathsf{Clauses}}(\mathsf{clauses}_1, \mathsf{clauses}_2)]\!] \\ &\subseteq [\![\mathsf{insts}_1]\!] \cup [\![\mathsf{clauses}_1]\!] = [\![t_1]\!] \end{aligned}$$

by Lemmas 38 and 44. □

Theorem 47 (TType Diff Satisfies (P2) Conditionally)**.** $\mathsf{diff}_{\mathsf{TType}}$ *satisfies (P2) when all underlying* $\mathsf{diff}_{\mathsf{STypes}}$ *calls satisfy (P2). In particular, (P2) holds when* STC *holds for all same-tag eliminations.*

Theorem 48 (TType Diff Completeness)**.** $\mathsf{diff}_{\mathsf{TType}}$ *is complete when:*

1. $\mathsf{diff}_{\mathsf{TInsts}}$ *is complete (structurally distinct instances are semantically disjoint), and*
2. $\mathsf{diff}_{\mathsf{STypes}}$ *is complete for all clause pairs (same-tag implies $[\![s]\!] \subseteq [\![s']\!]$).*

Corollary 49 (Explicit Completeness Conditions for $\mathsf{diff}_{\mathsf{TType}}$)**.** *Let $t_1 = (\mathsf{insts}_1, \mathsf{clauses}_1)$ and $t_2 = (\mathsf{insts}_2, \mathsf{clauses}_2)$. Then $\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)$ is complete (i.e. $[\![\mathsf{diff}_{\mathsf{TType}}(t_1, t_2)]\!] = [\![t_1]\!] \setminus [\![t_2]\!]$) under the following explicit hypotheses:*

1. ***Instance disjointness across sides:*** *for all* $i \in \mathsf{insts}_1$ *and* $j \in \mathsf{insts}_2$ *with* $i \neq j$*, we have* $[\![i]\!] \cap [\![j]\!] = \emptyset$*.*

2. ***Top-clause alignment:*** *if* $\mathsf{clauses}_1$ *is top (i.e. it contains the unique top clause* $(\emptyset, \mathsf{None})$*), then* $\mathsf{clauses}_2$ *is also top.*

3. ***Same-tag coverage for every clause subtraction:***

   (a) *For every* $k \in \mathrm{dom}(\mathsf{clauses}_1) \cap \mathrm{dom}(\mathsf{clauses}_2)$*, if* $\mathsf{clauses}_1(k) = (k, \mathsf{Some}(S_1))$ *and* $\mathsf{clauses}_2(k) = (k, \mathsf{Some}(S_2))$*, then* $\mathsf{STC}(S_1, S_2)$*.*

   (b) *If* $\emptyset \in \mathrm{dom}(\mathsf{clauses}_2)$ *and* $\mathsf{clauses}_2(\emptyset) = (\emptyset, \mathsf{Some}(S_0))$*, then for every* $k \in \mathrm{dom}(\mathsf{clauses}_1)$ *with* $\mathsf{clauses}_1(k) = (k, \mathsf{Some}(S_1))$*, we have* $\mathsf{STC}(S_1, S_0)$*.*

6 Application to Type Inference

Definition 50 (Subtyping Check with Validation)**.** *When checking* $u_1 \leq u_2$ *where* u_2 *contains type variable* T*:*

1. *Compute* $\mathsf{diff}(u_1, u_2)$ *and grow* T*'s lower bound.*

2. *Expand* u_1, u_2 *using the substitution.*

3. *Validate via* $\mathsf{isSub}(\mathit{expand}(u_1), \mathit{expand}(u_2))$*.*

Theorem 51 (Inference Soundness)**.** *If* isSub *correctly validates subtyping on expanded types, then type inference is sound regardless of whether* diff *satisfies (P2).*

Proof. The diff result is used only to guide inference (e.g. by accumulating bounds or hints about T). The acceptance/rejection decision is made solely by isSub on expanded types. Therefore, an imprecise diff can at worst cause incompleteness (e.g. by failing to discover a useful bound, leading to a later rejection), but it cannot cause an unsound acceptance when isSub is correct. □

7 Examples of Incompleteness

We present two concrete examples where diff is incomplete, i.e., $[\![\mathsf{diff}(t_1, t_2)]\!] \neq [\![t_1]\!] \setminus [\![t_2]\!]$ (it may be a strict subset or a strict superset of the true set difference).

7.1 Example 1: Same-Tag Literal Types

TypeScript code:

```
function f(x: 1 | 2): 2 {
  if (x === 1) { return 2; }
  return x;  // x should be narrowed to 2
}
```

Mathematical setup: Both `1` and `2` have the same runtime tag 4. Because STypes stores at most one SType per tag, it cannot precisely represent unions of same-tag literals such as `1 | 2`. In practice, the implementation merges same-tag literals into a single representative SType for the tag (often a wider type such as `number`), which loses the ability to subtract a single literal while keeping another.

Incompleteness: Since $\mathsf{tag}(\texttt{1}) = \mathsf{tag}(\texttt{2}) = 4$ and STypes stores at most one entry per tag, $\mathsf{diff}_{\mathsf{STypes}}$ cannot distinguish between literals of the same tag. For a simpler illustration:

$$\mathsf{diff}_{\mathsf{STypes}}(\{4 \mapsto \texttt{number}\}, \{4 \mapsto \texttt{1}\}) = \emptyset$$

But $[\![\texttt{number}]\!] \setminus [\![\texttt{1}]\!] =$ all numbers except $1 \neq \emptyset$.

The condition $\mathsf{STC}(L, R)$ fails: $[\![\texttt{number}]\!] \not\subseteq [\![\texttt{1}]\!]$.

7.2 Example 2: Top Type (unknown)

TypeScript code (requires `--strictNullChecks`):

```
function f(x: unknown): {} {
  if (x !== null && x !== undefined) {
    return x;  // x should be narrowed to {}
  }
  return {};
}
```

Mathematical setup: The type `unknown` is represented as a clause with $\mathsf{stypes} = \mathsf{None}$ (top). Let $c_1 = (\emptyset, \mathsf{None})$ represent `unknown`. Let c_2 represent `null | undefined`: $c_2 = (\emptyset, \mathsf{Some}(\{0 \mapsto \texttt{undefined}, 2 \mapsto \texttt{null}\}))$.

Incompleteness: By definition of $\mathsf{diff}_{\mathsf{Clause}}$:

$$\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2) = c_1 \quad (\text{since } \mathsf{stypes}_1 = \mathsf{None})$$

So $[\![\mathsf{diff}_{\mathsf{Clause}}(c_1, c_2)]\!] = [\![c_1]\!] = \mathsf{Val}$ (all values).

But the true set difference is:

$$[\![c_1]\!] \setminus [\![c_2]\!] = \mathsf{Val} \setminus ([\![\texttt{null}]\!] \cup [\![\texttt{undefined}]\!]) \neq \mathsf{Val}$$

The result is incomplete: diff returns `unknown` instead of the narrowed type $\{\}$ (non-nullish values). This is because the clause diff cannot "subtract from top"—it simply returns the left clause unchanged when $\mathsf{stypes}_1 = \mathsf{None}$.